

UNIVERSIDADE FEDERAL DO RECÔNCAVO DA BAHIA - UFRB  
CENTRO DE CIÊNCIAS EXATAS E TECNOLÓGICAS - CETEC

Projeto de Extensão

UFRBots - Uma visão extensionista para o futebol de robôs no Recôncavo da Bahia

Apostila do Minicurso de Simulação 2D do Futebol de Robôs



Apoio:



UNIVERSIDADE FEDERAL DO RECÔNCAVO DA BAHIA - UFRB  
CENTRO DE CIÊNCIAS EXATAS E TECNOLÓGICAS - CETEC

Autores:

Anderson dos Santos - Voluntário PIBEX  
Higor Santos de Jesus - Bolsista PIBEX  
João Augusto Moura Peixoto de Jesus - Voluntário PIBEX

Prof. André Luiz Carvalho Ottoni - Coordenador  
Prof. João Carlos Nunes Bittencourt  
Prof. Paulo Fábio Figueiredo Rocha

**Versão 1.1**

UNIVERSIDADE FEDERAL DO RECÔNCAVO DA BAHIA - UFRB  
CENTRO DE CIÊNCIAS EXATAS E TECNOLÓGICAS - CETEC

**Integrantes da UFRBots - Equipe de Futebol de Robôs da UFRB**

**Categoria Simulação 2D**

Anderson dos Santos  
Higor Santos de Jesus  
João Augusto Moura Peixoto de Jesus

**Very Small Size**

Ejziel Sampaio Santos  
Fabrício Velôso de Jesus  
Kevin Luis dos Santos  
Lucas Lopes Rios da Silva  
Mateus Nunes Tavares  
Samuel Rebouças de Jesus  
Walber Conceição de Jesus Rocha

**Orientadores**

Prof. André Luiz Carvalho Ottoni - Coordenador  
Prof. João Carlos Nunes Bittencourt  
Prof. Paulo Fábio Figueiredo Rocha

## SUMÁRIO

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>Aula 1 - Introdução ao Futebol de Robôs</b>         | <b>5</b>  |
| Definição                                              | 5         |
| Robótica Móvel                                         | 5         |
| Indústria 4.0                                          | 5         |
| Avanços na Robótica                                    | 5         |
| Futebol de Robôs                                       | 6         |
| RoboCup                                                | 6         |
| Categorias                                             | 6         |
| Áreas de Estudo                                        | 7         |
| Simulação 2D                                           | 7         |
| Simulador                                              | 7         |
| Projeto de Extensão                                    | 8         |
| Atividades                                             | 8         |
| <b>Aula 2 - 1ª Parte - Programação Básica</b>          | <b>9</b>  |
| Execução de passos                                     | 9         |
| Algoritmo                                              | 9         |
| Partes do algoritmo                                    | 10        |
| Fluxograma                                             | 11        |
| Tipos de dados                                         | 12        |
| Variável                                               | 12        |
| Comandos condicionais                                  | 12        |
| Repetição                                              | 12        |
| <b>Aula 2 - 2ª Parte - Programação Logo</b>            | <b>14</b> |
| Introdução                                             | 14        |
| Comandos básicos                                       | 14        |
| Projeto LOGO                                           | 15        |
| <b>Aula 3 - C++ - Estrutura de repetição e seleção</b> | <b>16</b> |
| Introdução                                             | 16        |
| Funções                                                | 17        |
| Condicionais                                           | 18        |
| Laço de Repetição                                      | 20        |
| <b>Aula 4 - c++ - Vetor, String, Struct e Função</b>   | <b>21</b> |
| Funções                                                | 21        |
| Vetor                                                  | 23        |
| Matriz                                                 | 24        |

|                                                  |           |
|--------------------------------------------------|-----------|
| String                                           | 24        |
| Funções Struct                                   | 27        |
| <b>Aula 5 - 1ª Parte - Comandos Linux</b>        | <b>29</b> |
| Abrir o terminal                                 | 29        |
| Abrir o monitor onde será vista a partida        | 29        |
| Preparar time para partida                       | 30        |
| Acessar pasta do time                            | 30        |
| Executar arquivo base do time                    | 31        |
| Compilar time                                    | 31        |
| Colocar time em campo                            | 31        |
| Iniciar partida                                  | 34        |
| <b>Aula 5 - 2ª Parte - Comandos 2D</b>           | <b>35</b> |
| Formação                                         | 35        |
| Mudança de Formação                              | 35        |
| Criar própria formação                           | 38        |
| <b>Aula 6 - Comandos 2D</b>                      | <b>42</b> |
| Chute                                            | 42        |
| Passes                                           | 43        |
| <b>Aula 7 - 1ª Parte - Preparação dos Times</b>  | <b>46</b> |
| Nome do Time                                     | 46        |
| Formação                                         | 46        |
| Chute                                            | 46        |
| Passe                                            | 47        |
| Jogar Partida                                    | 47        |
| <b>Aula 8 - 2ª Parte - Preparação dos Times</b>  | <b>48</b> |
| Parâmetros do Time                               | 48        |
| Jogar Partida                                    | 48        |
| Sorteio do Campeonato Final                      | 49        |
| <b>Instalação da Máquina Virtual</b>             | <b>50</b> |
| Download e Instalação do VirtualBox              | 50        |
| Download e Instalação da Máquina Virtual UFRBots | 50        |
| <b>Referências</b>                               | <b>53</b> |

|                                                                       |
|-----------------------------------------------------------------------|
| UFRBots                                                               |
| Uma visão extensionista para o futebol de robôs no Recôncavo da Bahia |
| <b>Aula 1 - Introdução ao Futebol de Robôs</b>                        |

## Definição

A robótica é um ramo da tecnologia que envolve diversas áreas de estudo, se destacando a mecânica, eletrônica e computação. Intrinsecamente ligada com a automação, permitindo a aplicação em diversos ramos da ciência [1].

## Robótica Móvel

A robótica móvel existe como um subgrupo da robótica, onde um agente atuante no ambiente, extrai e utiliza as informações através de sensores a fim de se deslocar com segurança de modo significativo e intencional, executando dessa forma diversas tarefas. Sua grande característica é a facilidade de locomoção e atuação em ambientes muitas das vezes desfavoráveis à robôs industriais, podendo se locomover com o auxílio de rodas, pernas, esteiras, entre outros [2].

## Indústria 4.0

Conhecida como a quarta revolução da indústria, tem como base um novo paradigma de produção. Onde o principal objetivo é integrar máquina e homem, otimizando e facilitando as atividades por meio de sistemas de controle, compondo de redes de entidades ao longo de todo o processo produtivo, transformando processos empíricos em autônomos permitindo personalização e controle de demandas individuais [3].

## Avanços na Robótica

A robótica é uma vasta área que proporciona avanços em diversos ramos da ciência, trazendo grandes e eficientes soluções para a astronomia, agricultura, oceanografia, medicina, indústria, lazer e em diversas atividades do cotidiano do homem.

## **Futebol de Robôs**

O futebol de robôs é uma tentativa de propiciar a pesquisa e inovação por meio de atividades relacionadas à robótica, inteligência artificial e campos correlacionados. Fomentado pelo objetivo principal de criar um time de robôs humanóides inteligentes, que possam competir com o time campeão da copa do mundo da fifa até 2050.

## **RoboCup**

A RoboCup é uma organização internacional, ligada ao futebol de robôs, que promove desenvolvimento na robótica e principalmente o futebol de robôs.

## **Categorias**

Como todo evento e organização possuem diversas categorias, a RoboCup não é diferente e dessa forma existem grupos distintos com objetivos específicos de pesquisa e atuação.

### **Humanoid**

São robôs com estrutura física similar a de um corpo humano, dos quais cada jogador além de possuir toda a estratégia em camp, também deve desempenhar todas funções motoras necessárias para se locomover, chutar ou até correr.

### **Standard Platform**

Similar a categoria humanoid, mas os robôs são iguais para os dois times, o robô NAO da SoftBank, dos quais jogam de forma totalmente autônoma e cada um toma decisões separadamente um do outro.

### **Middle Size**

Formada por equipes de cinco robôs totalmente autônomos e que utilizam uma bola de futebol com tamanho regular da FIFA.

### **Small Size**

É uma das mais antigas da competição e se concentra no problema de cooperação e controle inteligente multi-robô em um ambiente totalmente dinâmico, com times de seis robôs, onde todos os objetos do campo são rastreados por uma câmera localizada acima da superfície.

### **Simulation**

Dividida em dois grupos: 2D e 3D. O 2D com foco na aplicação de inteligência artificial, busca tratar todas as condições reais presentes nos jogos da FIFA e o 3D simula todas as condições da categoria *Standard Platform*.

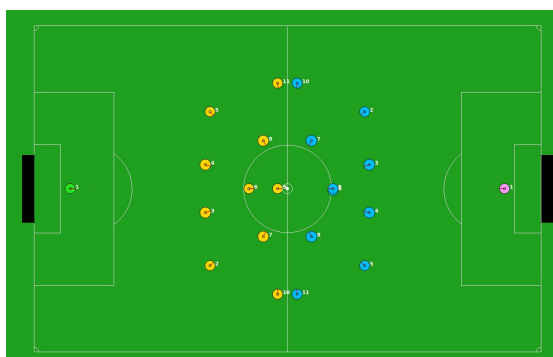
### **Áreas de Estudo**

As áreas de estudo propiciadas pelo futebol de robôs são diversas, das quais englobam: robótica móvel, visão computacional, inteligência artificial, sistemas embarcados, mecânica, eletrônica, e sistemas de controle.

### **Simulação 2D**

Essa será a categoria trabalhada ao longo do curso, onde dentre os objetivos cada equipe formará seu time. Essa simulação é composta de duas equipes com onze agentes autônomos que atuam em um campo bidimensional, onde são trabalhadas condições dos jogadores, bola e assim por diante.

**Figura 1:** Monitor do Simulador 2D



Fonte: Próprio autor

### **Simulador**

O simulador será utilizado para realizar as competições dos times desenvolvidos ao longo do curso, bem como na competição final. Para construção dos times serão trabalhados aspectos relacionados a: chute, passe, drible, corrida, formação e stamina.

## **Projeto de Extensão**

- Estimular o interesse dos estudantes pelas ciências exatas através do estudo do futebol de robôs.
- Incentivo aos futuros ingressantes na área de robótica.

## **Atividades**

Como principal atividade do projeto de extensão, estão as aulas expositivas relacionadas a programação básica e ao simulador 2D, bem como a preparação dos times para realizarem competições internas durante o curso e a final, que acontecerá no evento de seminário de robótica.

|                                                                       |
|-----------------------------------------------------------------------|
| UFRBots                                                               |
| Uma visão extensionista para o futebol de robôs no Recôncavo da Bahia |
| <b>Aula 2 - 1ª Parte - Programação Básica</b>                         |

### Execução de passos

Um computador pode realizar muitas tarefas em poucos instantes, das quais um ser humano poderia levar até dias para concluir. Contudo se faz necessária a definição de como ele deve executar e como seriam as etapas.

Uma das formas de definir esses processos com a programação, é seguindo passos que levem até a solução desejada, onde muitas das vezes remete a tomada de decisões.

**Exemplo:** No futebol de robôs, as jogadas devem ser pensadas de maneira que haja uma comunicação entre os jogadores, onde o objetivo principal seja marcar um placar. Dessa forma, pode ser definido os passos à serem executados para que um gol seja marcado.

#### *Solução:*

- 1) Direcionar o chute para outro jogador mais próximo do gol
- 2) Passar a bola para o outro jogador
- 3) Receber a bola
- 4) Dominar a bola
- 5) Driblar jogadores
- 6) Verificar se é possível chutar no gol
- 7) Chutar no gol
- 8) Marcar gol

**Exercício 01:** Como seria o passo a passo para você ir a escola, considerando que você estuda no turno matutino e o primeiro passo deve iniciar ao acordar.

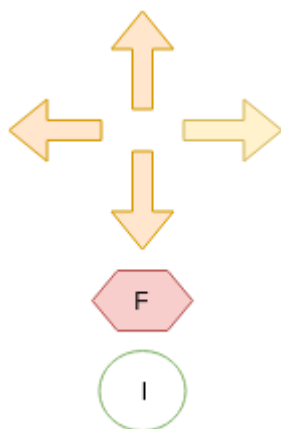
**Exercício 02:** Como seria o passo a passo para trocar uma lâmpada queimada, o primeiro passo deve tomar como base a identificação da lâmpada estar ou não queimada.

### Algoritmo

Um algoritmo se caracteriza pelo definição, ou seja, programação do passo a passo, podendo ser representado por um fluxograma, onde expressa-se tomadas de decisões e execução das tarefas.

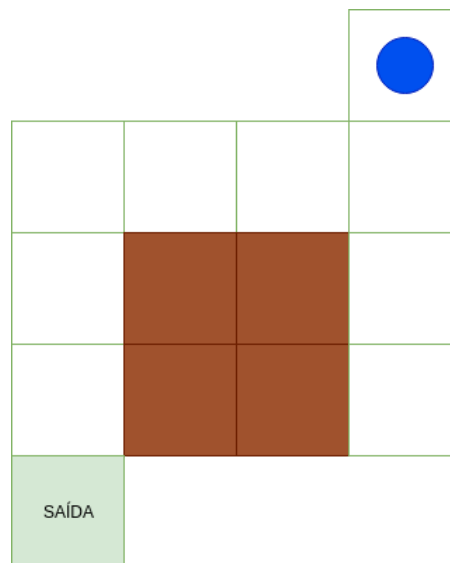
**Exemplo:** A bolinha azul do problema a seguir, representa um robô e precisa chegar até a saída. Quais seriam as entradas necessárias em ordem, para que a solução desejada ocorra?

**Figura 1 (a):** Entradas do Exemplo



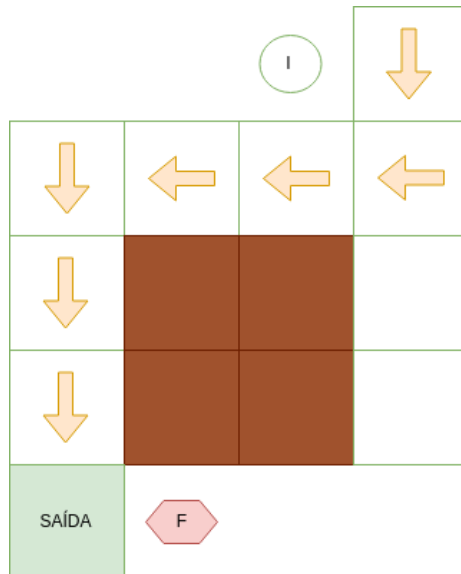
Fonte: Próprio Autor

**Figura 1 (b):** Problema do Exemplo



Fonte: Próprio Autor

**Figura 1 (c):** Solução do Exemplo



Fonte: Próprio autor

**Exercício 03:** É possível existir uma outra sequência de passos que leve ao mesmo resultado?  
Se sim, qual seria?

## Partes do algoritmo

Partindo do exemplo anterior, é possível inferir as partes fundamentais de um algoritmo:

- Início
- Entrada

Os dados são lidos ou fornecidos pelo usuário.

- Processamento

Lugar onde são tomadas as decisões, próximos passos, manipulando os dados de entrada onde o objetivo principal é chegar em um resultado, que é a saída (próxima parte do algoritmo).

- Saída

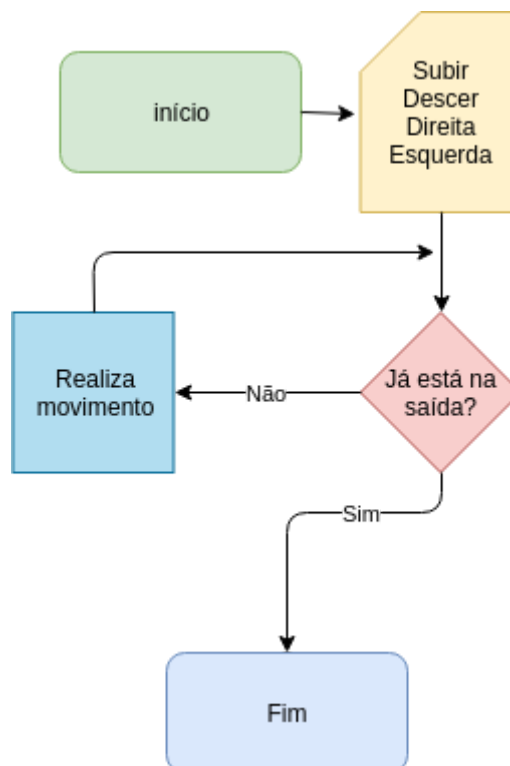
São basicamente o(s) resultado(s) do processamento.

- Fim

## Fluxograma

O fluxograma, segue a lógica dos passos de execução e as partes de um algoritmo. Utilizando o exemplo do robô, temos o seguinte fluxograma.

**Figura 2:** Fluxograma do problema do robô



Fonte: Próprio autor

**Exercício 04:** Construa um fluxograma com base no exercício 02.

**Exercício 05:** Construa um fluxograma com base nas decisões de um jogador em campo. Levando em consideração a decisão de tocar a bola e chutar no gol.

### Tipos de dados

- Inteiro
  - guarda valores inteiros numéricos.
  - permite fazer operações.
- Real
  - guarda valores numéricos e decimais .
  - espaço reservado maior do que para inteiro.
- Caracter
  - guarda qualquer coisa, seja letra ou número.
  - não permite fazer operações.
- Lógico
  - só aceita verdadeiro ou falso.

### Variável

Uma variável, pode ser entendida como uma gaveta de uma cômoda, onde cada gaveta guarda objetos divididos por categoria, sendo essas categorias os tipos de dados vistos anteriormente.

### Comandos condicionais

Tomadas de decisão, remete a uma verificação de determinadas entradas ou resultados de processamentos.

Condiciona l “se”:

- Uma etapa do algoritmo é executada caso seja válida a verificação
- Voltando para o exemplo do fluxograma, o losango representa um comando condicional, onde a partir da resposta para a pergunta, é decidido qual caminho tomar e como resposta positiva a execução se direciona para o fim do algoritmo.

Condiciona l “se não”:

- Se configura como a condição contrária ao “se”, ou seja, é a etapa do algoritmo que é executada a receber uma resposta negativa.

- Continuando no exemplo do fluxograma, se configura o “se não”, quando o robô ainda não está na saída e o algoritmo se direciona para realizar um novo movimento.

## Repetição

Uma estrutura de repetição, se configura como algo que se repete várias vezes até que uma condição seja estabelecida.

- Por exemplo, no exemplo do fluxograma o processamento de realizar um novo movimento é executado várias vezes, até que o robô chegue até a saída.
- Onde poderia ser descrita dessa forma:
  - enquanto (o robô não chegue a saída)

|                                                                       |
|-----------------------------------------------------------------------|
| UFRBots                                                               |
| Uma visão extensionista para o futebol de robôs no Recôncavo da Bahia |
| <b>Aula 2 - 2ª Parte - Programação Logo</b>                           |

## Introdução

Linguagem intuitiva, onde o objetivo principal se resume em controlar os movimentos de uma tartaruga que representa um robô. Onde por meio de comandos é possível definir a trajetória do robô e criar desenhos.

## Comandos básicos

→ Legenda:

◆  $n^{\circ}$  - número inteiro de entrada

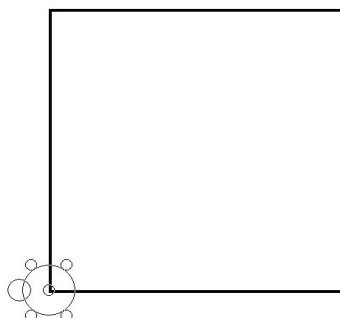
- **parafrente  $n^{\circ}$  ou pf  $n^{\circ}$** 
  - faz com que a tartaruga caminhe para frente  $n^{\circ}$  passos
- **paratrás  $n^{\circ}$  ou pt  $n^{\circ}$** 
  - faz com que a tartaruga caminhe para trás  $n^{\circ}$  passos
- **paradireita  $n^{\circ}$  ou pd  $n^{\circ}$** 
  - Gira a tartaruga para a direita em  $n^{\circ}$  graus
- **paraesquerda  $n^{\circ}$  ou pe  $n^{\circ}$** 
  - Gira a tartaruga para a esquerda em  $n^{\circ}$  graus
- **tartaruga ou tat**
  - apaga a tela e coloca a tartaruga na posição inicial (0,0)

**Exemplo:** Desenhar com a tartaruga um quadrado de lados tamanho 100.

**Solução:**

- pf 100
- pd 90
- pf 100
- pd 90
- pf 100
- pd 90
- pf 100

**Figura 1:** Desenho feito pela tartaruga



Fonte: Próprio Autor

## Projeto LOGO

A programação LOGO possui várias distribuições, das quais podem ser encontradas no site ProjetoLogo<sup>1</sup>, que é dedicado a descrever sua utilização, bem como a disposição das distribuições disponíveis para uso.

A distribuição utilizada para essa aula foi a *SuperLogo*.

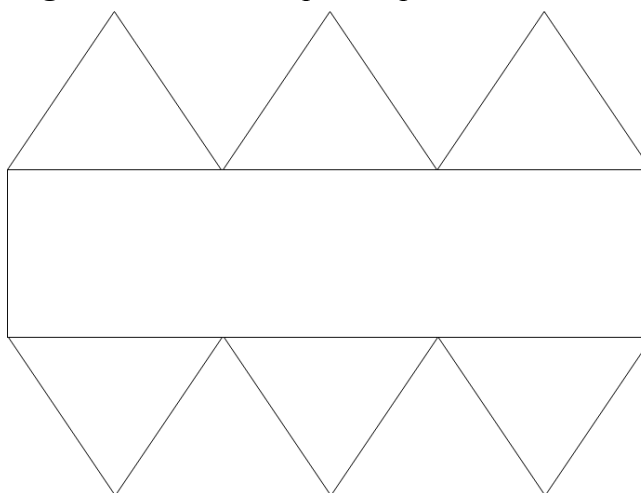
Site do ProjetoLogo : [projetologo.webs.com](http://projetologo.webs.com)

Acessando o site disposto acima, é possível realizar o *download* de uma das distribuições e utilizar em seu próprio computador.

## Exercícios

01. Desenhe com a tartaruga um triângulo, onde cada lado tenha tamanho igual a 100.
02. Construindo uma combinação de figuras. Tomando como base a criação de um triângulo e um retângulo, faça 6 triângulos e um retângulo, de modo que fiquem dispostos 3 triângulos na parte superior e 3 na parte inferior do retângulo, lembrando que os triângulos devem ter a mesma dimensão e o retângulo deve comportar de forma exata a extensão de 3 triângulos em seu comprimento, como na figura abaixo:

**Figura 2:** Desenho requerido para o exercício 02



Fonte: Próprio Autor

03. Construa a fachada de uma casa, contendo uma porta, uma janela e telhado.
04. Desafio: Crie somente os comandos (sem executar inicialmente) para o desenho de um losango.

---

<sup>1</sup> Site do ProjetoLogo: [projetologo.webs.com](http://projetologo.webs.com)

|                                                                       |
|-----------------------------------------------------------------------|
| UFRBots                                                               |
| Uma visão extensionista para o futebol de robôs no Recôncavo da Bahia |
| <b>Aula 3 - C++ - Estrutura de repetição e seleção</b>                |

## Introdução

C é uma das linguagens de programação mais populares e existem poucas arquiteturas para as quais não existem compiladores para C. C tem influenciado muitas outras linguagens de programação (por exemplo, a linguagem Java), mais notavelmente C++, que originalmente começou como uma extensão para C. A linguagem C encontra-se na versão/padrão internacional C18 standard revision (ou ISO/IEC 9899:2018) lançada em junho de 2018, substituindo a versão C11 (standard ISO/IEC 9899:2011), disponível em ISO e IEC e com suporte para GCC8 e Clang LLVM6.

C++ é uma linguagem de programação compilada multi-paradigma(seu suporte inclui linguagem imperativa, orientada a objetos e genérica) e de uso geral. Desde os anos 1990 é uma das linguagens comerciais mais populares, sendo bastante usada também na academia por seu grande desempenho e base de utilizadores. Bjarne Stroustrup desenvolveu o C++ (originalmente com o nome *C with Classes*, que significa *C com classes* em português) em 1983 no Bell Labs como um adicional à linguagem C. Após a padronização ISO realizada em 1998 e a posterior revisão realizada em 2003, uma nova versão da especificação da linguagem foi lançada em dezembro de 2014, conhecida informalmente como C++.

Temos como exemplos de programas e usos diretos da linguagem C/C++ todas as versões do sistema operacional windows, que é utilizado na maior parte dos computadores nos dias atuais. Além disto o sistema operacional Linux também foi escrito em linguagem C

## Vantagens:

- É uma linguagem simples que nos permite trabalhar com funções matemáticas, ficheiros, entre outras sendo necessário para tal a inclusão de bibliotecas padrão as quais já vem com a linguagem;
- Por haver compiladores C para quase todos os computadores, é possível tomar um código escrito para uma máquina, compilá-lo e rodá-lo em outra com pouca ou nenhuma modificação. Essa "portabilidade" é uma das características do C (Um

compilador lê o programa inteiro e converte-o em um código-objeto, que é uma tradução do código-fonte do programa em uma forma que o computador possa executar diretamente. Desta maneira, tudo que o programador precisa fazer para executar o programa é, normalmente, digitar apenas o seu nome);

- Tipos de dados simples;
- Acesso fácil ao hardware, baixos requisitos de memória;
- Definição e declaração simples de variáveis.

### Desvantagens:

- Os programas criados em C, em termos de segurança não são muito seguros uma vez que o código é escrito de uma forma seqüencial (pode-se facilmente incluir uma instrução no nosso código do programa e tornar o resultado final completamente diferente);
- Não possui coleta automática de lixo (se durante o programa uma variável declarada não for mais usada ao longo do programa, a linguagem não tem capacidade para eliminá-la continuando esta a ocupar espaço indevidamente em memória);
- Não faz uso de classes ou objetos (como no JAVA e C++);
- Networking (é uma linguagem que inicialmente não foi pensada para redes).

Para começo de estudo, vamos considerar um detalhe das linguagens de programação chamado **Palavras reservadas**, as palavras reservadas são expressões únicas de cada linguagem de programação e para esta aula em C/C++ conheceremos algumas dessas palavras e seus usos básicos. Tais palavras serão divididas em três subgrupos: Funções, Condicionais e Repetição/Loops. Para testes e exercícios vamos utilizar o compilador (IDE) Dev-C++.

### Funções

**Printf:** O primeiro argumento da função printf corresponde ao endereço de uma cadeia de caracteres. Esta cadeia de caracteres será lida e os caracteres serão escritos no terminal (ou console) a menos que estes caracteres sejam identificados como sendo especificações de conversão.

Exemplo: (1):

**Figura 1:** Exemplo da função printf no Dev-C++

```
1  #include<stdio.h>
2  int main(){
3      printf("Bem vindos ao minicurso de futebol 2D da equipe UFRBots\n");
4      return 0;
5  }
```

Fonte:Próprio autor

**Scanf:** A função scanf opera, de forma similar à printf, sobre um fluxo de caracteres, mas agora trata-se de um “fluxo de entrada”. Os caracteres obtidos na “entrada” (em geral a scanf opera sobre um fluxo de caracteres oriundo do teclado), devem ser tratados (ou convertidos e atribuídos aos demais argumentos) de acordo com os caracteres do primeiro argumento.

**Exemplo:**

**Figura 2:** Exemplo da função scanf no Dev-C++

```
1  #include<stdio.h>
2  int main()
3  {
4      printf(" Digite um numero de 1 a 100\n");
5      scanf("%d",&x);
6      printf("O numero foi: %d", x);
7      return 0;
8  }
```

Fonte:Próprio autor

Estruturas condicionais são um pouco mais complexas em relação ao código do que os exemplos anteriores, porém são de fácil entendimento quando são analisadas com base no que suas **palavras reservadas** significam.:

## Condicionais

**If e else :** A estrutura simples contém apenas uma condição. Se esta condição for verdadeira o programa executa sua(s) instrução(ões).

### Exemplo:

Figura 3: Exemplo da função if e else no Dev-C++

```
1  #include<stdio.h>
2  int main(){
3      int x;
4      printf(" Digite um numero de 1 a 100\n");
5      scanf("%d",&x);
6      if(x%2==0){
7          printf("O numero foi: %d e ele é par", x);
8      }else{
9          printf("O numero foi: %d e ele é impar", x);
10     }
11     return 0;
12 }
13
```

Fonte:Próprio autor

**OBS:** A expressão  $(x\%2==0)$  exemplificada no **If** significa que: se o resto da divisão (%) de  $x$  por 2 for igual a 0 o número é par, senão (**else**) for igual a 0 ele é ímpar. Note também que quando o código for executado o caractere é não aparecerá na tela, pois a biblioteca básica que estamos utilizando não tem suporte para palavras acentuadas.

**Switch** : A Instrução switch efetua vários testes de valores sobre o conteúdo de uma mesma variável. Esta conexão condicional simplifica muito o teste de vários valores de uma variável, pois esta operação teria sido complicada (mas possível) com **if's** sequenciados, parênteses após a palavra-chave *switch* indicam uma expressão cujo valor é testado, sucessivamente, para cada uma das *case*. Quando a expressão testada é igual a um dos seguintes valores seguindo um *case*, a lista de instrução que vem depois, é executada. A palavra-chave *break* indica a saída da estrutura condicional. A palavra-chave *default* precede a lista de instruções, que será executada se a expressão nunca for igual a um dos valores.

Não se esqueça de inserir instruções *break* entre cada teste; esse tipo de esquecimento é difícil de ser detectado pois nenhum erro é assinalado. Na verdade, ao omitir o *break*, a execução continuará nos blocos seguintes.

### Exemplo:

Figura 4: Exemplo da função switch no Dev-C++

```
1  #include<stdio.h>
2  int main(){
3      int x,y,r;
4      char c;
5      printf(" Digite dois numeros inteiros\n");
6      scanf("%d %d",&x,&y);
7      printf("Escolha uma das operações (+,-,*,/)");
8      scanf("%c",&c);
9      switch(c){
10         case '+':
11             r=(x+y);
12             printf("O resultado da soma foi %d",r);
13             break;
14         case '-':
15             r=(x-y);
16             printf("O resultado da subtração foi %d",r);
17             break;
18         case '*':
19             r=(x*y);
20             printf("O resultado da multiplicação foi %d",r);
21             break;
22         case '/':
23             r=(x/y);
24             printf("O resultado da divisão foi %d",r);
25             break;
26         default:
27             printf("Operação invalida");
28     }
29 }
30 return 0;
31
```

Fonte:Próprio autor

As estruturas de repetição nos permitem fazer mais coisas assim como as estruturas de seleção, porém com menos linhas de código.

### Laço de Repetição:

**For:** A instrução **for** executa várias vezes a mesma série de instruções: é um loop. Em sua sintaxe, basta especificar o nome da variável, que serve de contador (e eventualmente o seu valor inicial, a condição sobre a variável para a qual o loop para (basicamente uma condição que testa se o valor do contador excede um limite), e finalmente uma instrução que incrementa (ou decrementa) o contador.

**While :** A instrução **while** representa outro meio de executar várias vezes a mesma série de instruções. A sintaxe desta expressão será demonstrada no exemplo a seguir.

### Exemplo:

Figura 5: Exemplo da funções while e for no Dev-C++

```
1  #include<stdio.h>
2  int main(){
3      int i, numero;
4      printf(" Digite um numero da tabuada\n");
5      scanf("%d",&numero);
6      printf("\n\nVersão com while");
7      i=1;
8      while(i<=10){
9          printf("\n%d x %d = %d",numero,i,numero*i);
10         i++;
11     }
12     printf("\n\nVersão com for");
13     for(i=1;i<=10;i++){
14         printf("\n%d x %d = %d",numero,i,numero*i);
15     }
16     return 0;
17 }
```

Fonte:Próprio autor

### Exercícios:

- 1) Modifique o exemplo 1 de forma que exiba uma mensagem de sua escolha.
- 2) Modifique o exemplo 2 de forma que o programa receba dois números inteiros.
- 3) Utilizando **Ifs e elses ou switch** jogue pedra, papel e tesoura.
- 4) Utilize como base o programa descrito no exemplo 2. Modifique o código do exemplo 3 para que o programa receba 2 números e mostre o maior deles na tela.
- 5) Utilizando o código da questão anterior faça com que os números sejam mostrados na tela de forma crescente.
- 6) Modifique o exemplo 4 para que o mesmo faça a seleção entre quatro objetos de sua preferência com base nos caracteres (^,-,\$,@).
- 7) Modifique o exemplo 5 para que mostre seu nome na tela 20 vezes. (Tanto com o **for** quanto com o **while**).
- 8) Dados valores de X e Y indique se um jogador está no ataque, defesa ou meio de campo.
- 9) Incremente o exercício anterior de forma apareça na tela as mensagens:
  - a) (“CHUTE AO GOL”) quando o jogador estiver no ataque.
  - b) (“CARRINHO”) quando o jogador estiver na defesa.
  - c) (“TOQUE DE BOLA”) quando o jogador estiver no meio de campo.

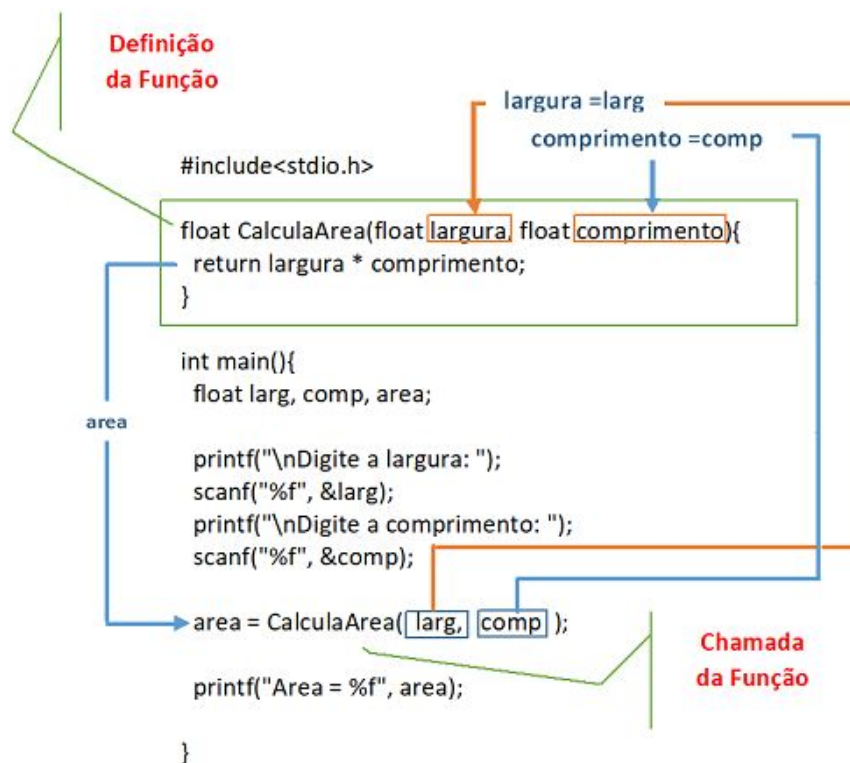
|                                                                       |
|-----------------------------------------------------------------------|
| UFRBots                                                               |
| Uma visão extensionista para o futebol de robôs no Recôncavo da Bahia |
| <b>Aula 4 - c++ - Vetor, String, Struct e Função</b>                  |

Como continuação da aula passada, vamos agora desenvolver a linguagem C de forma mais complexa. Para relembrarmos a aula passada vamos considerar um detalhe das linguagens de programação chamado **Palavras reservadas**, as palavras reservadas são expressões únicas de cada linguagem de programação e para esta aula em C/C++ conheceremos algumas dessas palavras e seus usos básicos. Tais palavras serão divididas em quatro subgrupos: Funções, Vetores/Arrays, Strings e Structs. Para testes e exercícios vamos utilizar o compilador (IDE) Dev-C++.

### Funções

- Uma função pode ser vista como um conjunto de comandos que realiza uma tarefa específica. Em outras palavras, pode-se dizer que é um pequeno "programa" utilizado por outros programas.
- A função é referenciada (chamada) pelo programa principal através de um nome atribuído a ela.
- A utilização de funções, muito comum na programação estruturada, visa subdividir um programa em partes (módulos) menores que realizam uma tarefa bem definida.
- Benefícios da utilização de funções:
  - Permite o reaproveitamento de código já construído (por você ou por outros programadores);
  - Evita que um mesmo trecho de código seja repetido várias vezes dentro de um mesmo programa e, com isso, qualquer alteração é feita apenas nesse trecho e de forma simples.
  - Para que os blocos do programa não fiquem grandes demais e, por consequência, mais difíceis de entender;
  - Facilita a leitura do programa de maneira que os blocos de código possam ser logicamente compreendidos de forma isolada.

**Figura 1:** Exemplo de função em C, Prof. Marcos Vaz



Fonte: Vaz, Marcos. Apostila de apresentação a Linguagem C. CodeinGame, 2019.

Disponível

em: <<https://www.codingame.com/playgrounds/24988/programacao-c/funcoes-em-c>>. Acesso em: 20 de ago. de 2019.

### Parâmetros:

- A Lista de Parâmetros, também é chamada de Lista de Argumentos, é opcional. Funcionam como a interface de comunicação (passagem de valores/dados) entre o programa (chamador) e a função.
- Os parâmetros de uma função são definidos como se estivesse declarando uma variável, entre os parênteses do cabeçalho da função. Caso precise declarar mais de um parâmetro, basta separá-los por vírgulas.

Existem 2 maneiras de passar valor através dos parâmetros: Por valor ou Por referência. Nesse momento, será apresentado a passagem de parâmetro por valor. Quando introduzirmos o conceito de Ponteiro, apresentaremos o outro de tipo.

**OBS:** Os parâmetros são passados para uma função de acordo com a sua posição. Ou seja, o primeiro parâmetro da chamada (programa) define o valor o primeiro parâmetro na definição da função, o segundo parâmetro do programa define o valor do segundo parâmetro da função e assim por diante. Os nomes dos parâmetros na chamada não tem relação com os nomes dos parâmetros na definição da função.

### Passagem de Parâmetros por valor:

- A passagem de parâmetros que será apresentada aqui é chamada de passagem por valor.
- Os valores contidos nas variáveis do programa, no momento da chamada da função, são copiados para as variáveis parâmetros da função.
- As alterações dos valores dos parâmetros dentro da função não afetarão os valores das variáveis do programa, usadas na chamada da função.
- Quando uma função não for retornar valor (procedimento), utilize void como tipo da função. Quando se usa void não podemos usar return na função.

### Vetor

Arrays representam em matemática vetores e matrizes. Em C tanto vetores quanto matrizes são implementados usando uma variável composta de várias posições indexadas numericamente.

#### Exemplo:

```
int Vetor[6]; /* declara um vetor do tipo int (números inteiros) de 6 posições */  
float Vetor[3]; /*declara um vetor do tipo float(números com casas decimais) com 3 posições */  
char Vetor[10] /* declara um vetor do tipo char com 10 posições */
```

Na linguagem C, os vetores têm o primeiro elemento na posição 0(zero). Assim, se tomarmos "K" como sendo o tamanho do vetor a última posição é a de índice "K-1". Por exemplo, um vetor de tamanho 10 (k=10) tem o seu último elemento na posição 9. Para inicializar um vetor, isto é, mover valores para todos os elementos precisamos de uma estrutura de repetição. A estrutura de repetição FOR se adequa perfeitamente às Matrizes(vetores) porque a variável de controle da repetição pode ser usada para marcar as posições do vetor.

#### Exemplo: Inicialização de vetor

Figura 2: Exemplo de vetor no DevC++

```
1  #include <stdio.h>  
2  int main() {  
3  
4      int V[10],i;  
5  
6      for(i=0;i<10;i++){  
7          if (i%2 !=0){  
8              V[i]=0;  
9          }  
10         else {  
11             V[i]=1;  
12         }  
13     }  
14     return 0;  
15 }
```

Fonte: Próprio autor

## Matriz

As matrizes são estruturas de dados que possuem mais de uma dimensão (vetor). Diferentemente dos vetores que precisam de apenas um índice para acesso aos elementos, as matrizes necessitam utilizar um índice para cada dimensão.

### Exemplo:

```
int Matriz[6][2]; /* declara uma matriz do tipo int (números inteiros) com 6 linhas e 2 colunas */
```

```
float Matriz[5][3]; /*declara uma matriz do tipo float(números com casas decimais) de 5 linhas e 3 colunas(15 elementos) */
```

```
char Matriz[2][10] /* declara uma matriz do tipo char com 2 linhas e 10 colunas */
```

Para inicializar uma Matriz, isto é, mover valores para todos os elementos precisamos de duas estrutura de repetição encadeadas. Uma estrutura de repetição **FOR** para percorrer as linhas da Matriz e outra para percorrer as colunas.

### Exemplo: Inicialização de matriz

**Figura 3:** Exemplo de matriz com a utilização da função random no DevC++

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  int main() {
4
5      int i, j, n = 0, v[6][6];
6
7      printf("\nMatriz preenchida aleatoriamente: \n");
8      for (i = 0 ; i < 6 ; i++){
9          for (j = 0 ; j < 6 ; j++){
10             v[i][j] = (rand() % 100);
11             printf(" %d ", v[i][j]);
12         }
13         printf("\n");
14     }
15
16     return 0;
17 }
```

Fonte: Próprio autor

## String

Uma cadeia de caracteres (string em inglês) é uma sequência de caracteres ou símbolos. Em linguagem C, uma cadeia de caracteres pode ser tratada como um "Vetor" do tipo char, cujo último elemento é o caractere nulo '\0'. A declaração de uma string em C é semelhante a declaração de um vetor do tipo char. O número máximo de caracteres que comporta a cadeia será igual ao número de elementos do vetor menos um (reservado para o caractere do final da

cadeia), por exemplo, se definirmos uma cadeia de caracteres de tamanho 30, ela conterá 29 caracteres + '\0'.

### **char Nome\_da\_tabela[Número\_de\_elementos]**

- A inicialização de uma string, ou seja, mover caracteres, pode ser feita de duas maneiras:
  - Mover manualmente os dados (célula por célula), como em um vetor ou
  - Utilizar funções de manipulação de cadeias fornecidas pela biblioteca padrão (String.h).

Exemplo de inicialização manual de cadeia de caracteres (para impressão de cadeia de caracteres com o comando printf devemos utilizar o caracter formatação "%s").

**OBS:** O comando scanf não lê o caracter Enter('\n'), portanto em variáveis do tipo char esse caracter é lido no comando scanf seguinte. Logo, em comandos de leitura de variáveis do tipo char, coloque a função getch() ou fflush(stdin) após o comando scanf, para solucionar o problema.

### **Função gets – (get string)**

Esta função armazena tudo que foi digitado, inclusive os espaços, até que a tecla ENTER seja pressionada.

**Figura 4:** Exemplo de string com scanf no DevC++

```
1  #include<stdio.h>
2  #include<conio.h>
3  int main(void)
4  {
5      char nome[61];
6
7      printf("Digite seu nome: ");
8      scanf("%s", nome);
9
10     printf("O nome armazenado foi: %s", nome);
11
12     getch();
13     return 0;
14 }
```

Fonte: Próprio autor

Figura 5: Exemplo de string com gets no DevC++

```
1  #include<stdio.h>
2  #include<conio.h>
3  int main(void)
4  {
5      char nome[61];
6
7      printf("Digite seu nome: ");
8      gets(nome);
9
10     printf("O nome armazenado foi: %s", nome);
11
12     getch();
13     return 0;
14 }
```

Fonte: Próprio autor

Com o começo de uso das **Strings**, também vamos começar a utilizar uma nova biblioteca. A movimentação de valores de/para as cadeias de caracteres não pode ser efetuadas diretamente pelo comando de atribuição, ou pode ser efetuada pela movimentação caracter a caracter como em vetores ou através de funções de manipulação de cadeias de caracteres. Estas funções estão no cabeçalho do arquivo **<string.h>**. Nesse momento, apresentaremos apenas as funções mais importantes de manipulação de Strings.

- **Mover caracteres para uma variável string.**

A movimentação pode ser efetuada caracter a caracter (como em um vetor) ou através da função `strcpy` (string copy).

`char *strcpy (const char *dest, const char *src)` - Copia uma string de `src` para outra string `dest`.

- **Comparação de Strings.**

A comparação pode ser efetuada caracter a caracter (como em um vetor) ou através da função `strcmp` (string compare).

`int strcmp(const char *string1, const char *string2)` - Compara duas strings (`string1` e `string2`) e determina a ordem (alfabética) das duas.

- **Tamanho da cadeia de caracteres**

Informa a quantidade de caracteres (tamanho) em uma cadeia de caracteres, excetuando o caractere de fim da string('\0').

**int strlen(const char \*string)** - Retorna o tamanho de uma string.

- **União (Concatenação) da cadeia de caracteres**

Concatena (une) caracteres de duas cadeias de caracteres em uma string.

**char \*strcat(const char \*string1, char \*string2)** - Concatena os caracteres da string2 na string1.

### Funções Struct

- As estruturas de dados (Homogêneas e Heterogêneas) possibilitam a construção de estruturas mais complexas que os tipos de dados primitivos (int, char,...).
- Diferentemente dos tipos homogêneos, essas estruturas permitem a manipulação de um conjunto de informações de tipos de dados primitivos diferentes, mas que possuem um relacionamento lógico entre si;
- Exemplo de um registro de Funcionário

**Figura 6:** Exemplo de construção de struct.

| Registro Funcionário |                           |
|----------------------|---------------------------|
| Matricula            | Tipo Inteiro              |
| Nome                 | Tipo Cadeia de Caracteres |
| Dt Nascimento        | Tipo Data                 |
| Cargo                | Tipo Cadeia de Caracteres |
| Salário              | Tipo Real                 |

Fonte: Vaz, Marcos. Apostila de apresentação a Linguagem C. CodeinGame, 2019. Disponível

em:<<https://www.codingame.com/playgrounds/24988/programacao-c/funcoes-em-c>>. Acesso em: 20 de ago. de 2019.

- O Registro acima possui um conjunto informações relacionadas a um funcionário, logo poderiam ser vistas agrupadas num único nome (como as Matrizes).
- A linguagem C possui uma estrutura denominada **struct** que permite agrupar um conjunto de informações de tipos diferentes sob um mesmo nome.

### **Exercícios:**

- 1) Construa um programa que crie um vetor V de 10 elementos do tipo inteiro. Mostrar V.
- 2) Construa um programa que leia uma matriz 6x6, dada pelo usuário, conte e escreva quantos valores maiores que 10 ela possui (Utilize a função random() para preencher a matriz com valores aleatórios).
- 3) Construa um programa que leia uma matriz A (6x5) do tipo inteiro e construa uma matriz B de mesmo tipo e dimensão. Sendo que B deve ser formada do seguinte modo: a cada elemento par da matriz A deve ser somado 5 e de cada elemento ímpar da matriz A deve ser subtraído 4. No final mostrar as matrizes A e B.
- 4) Construa um vetor que leia 10 elementos do tipo inteiro e mostre o maior e o menor entre eles.

## UFRBots

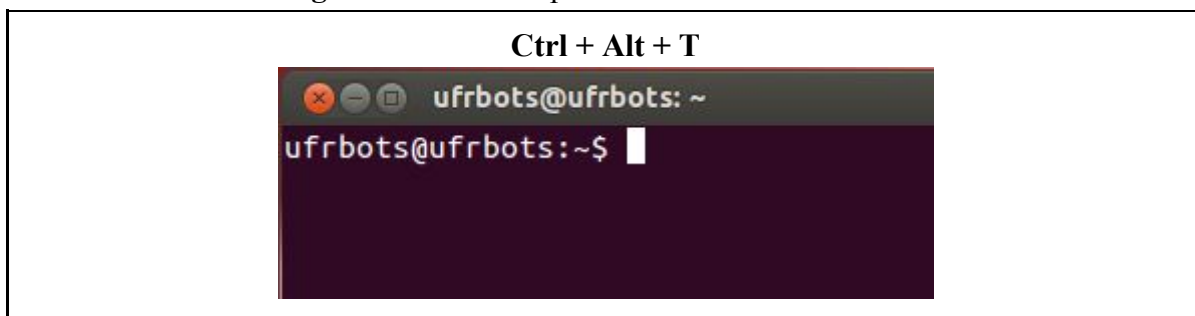
Uma visão extensionista para o futebol de robôs no Recôncavo da Bahia

### Aula 5 - 1ª Parte - Comandos Linux

Os comandos listados a seguir, servem para a utilização básica do simulador, sendo para acessar os arquivos dos times que serão utilizados, bem como outras atividades. Seguem os passos para inicial um time.

#### Abrir o terminal

**Figura 1:** Comando para abrir o terminal no linux

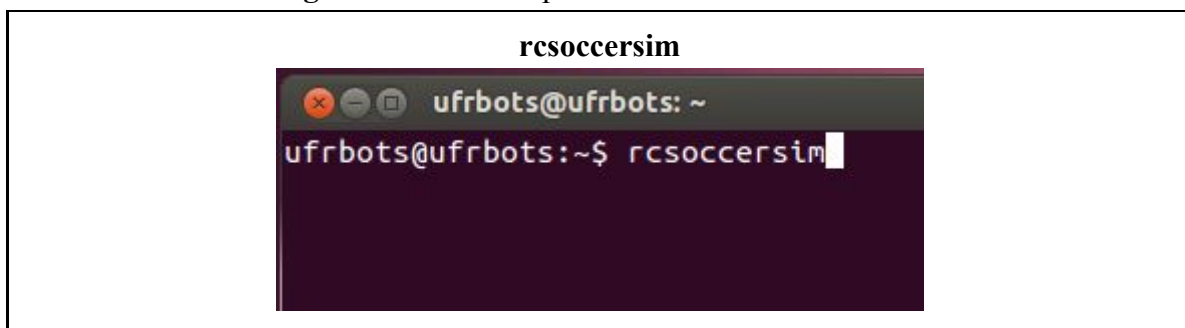


Fonte: Próprio autor

#### Abrir o monitor onde será vista a partida

Em um novo terminal (*passo 1*) executar o próximo comando para iniciar a tela de visualização do simulador.

**Figura 2:** Comando para iniciar a tela do monitor





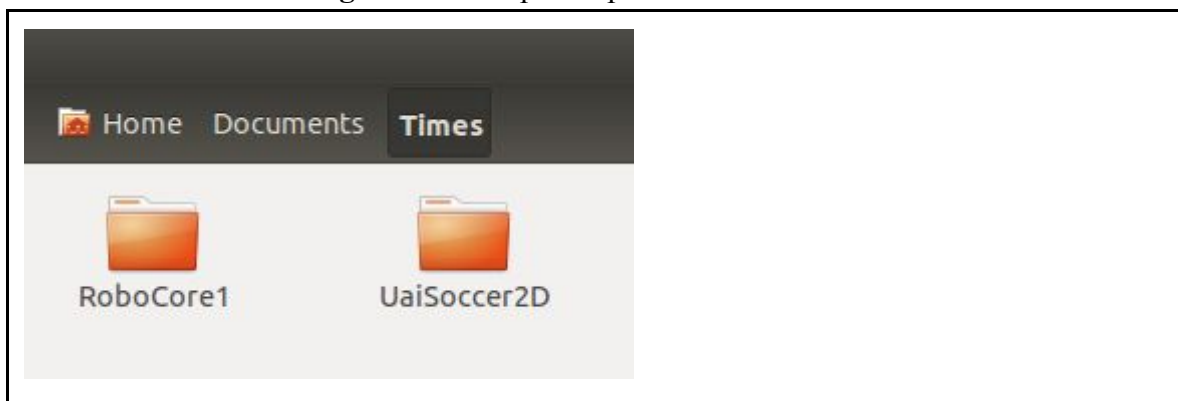
Fonte: Próprio autor

## Preparar time para partida

### Exemplo de times

Sejam os seguinte times presentes na pasta *Times*. O primeiro time a ser iniciado será o *RoboCore1*.

**Figura 3:** Exemplo de pastas de dois times



Fonte: Próprio autor

## Acessar pasta do time

Primeiro vamos acessar a pasta do time *RoboCore1*.

**Figura 4:** Comando para acesso a pasta



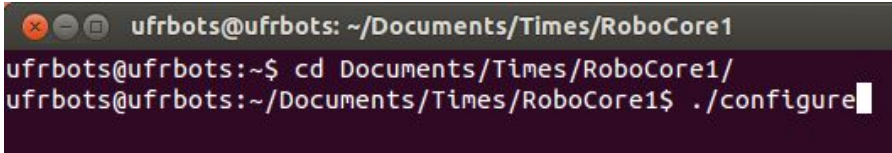
Fonte: Próprio autor

## Executar arquivo base do time

Após acessar a pasta, pelo passo anterior, deve-se executar o arquivo base do time para compor pré-definições.

**Figura 5:** Comando para executar arquivo base

```
./configure
```



```
ufrbots@ufrbots: ~/Documents/Times/RoboCore1  
ufrbots@ufrbots:~$ cd Documents/Times/RoboCore1/  
ufrbots@ufrbots:~/Documents/Times/RoboCore1$ ./configure
```

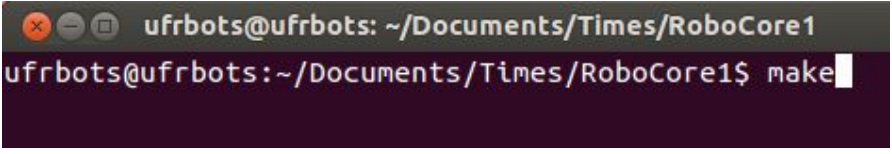
Fonte: Próprio autor

## Compilar time

O passo seguinte é muito importante e deve ser executado em seguida ao passo anterior. Aqui o time será compilado para ser iniciado.

**Figura 6:** Comando para compilar arquivos do time

```
make
```



```
ufrbots@ufrbots: ~/Documents/Times/RoboCore1  
ufrbots@ufrbots:~/Documents/Times/RoboCore1$ make
```

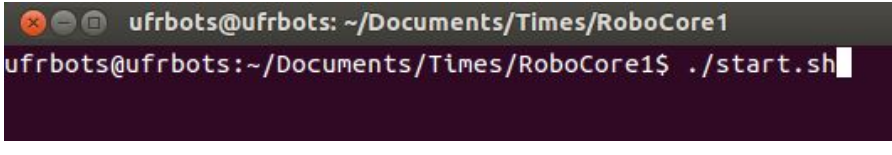
Fonte: Próprio autor

## Colocar time em campo

A próxima etapa se resume em iniciar o time, ou seja, prepará-lo em campo no simulador para uma nova partida. Dessa forma o time *RoboCore1* entra em campo e fica aguardando seu adversário.

**Figura 7:** Comando para iniciar time

```
./start.sh
```



```
ufrbots@ufrbots: ~/Documents/Times/RoboCore1  
ufrbots@ufrbots:~/Documents/Times/RoboCore1$ ./start.sh
```



Fonte: Próprio autor

**Repetir o mesmo processo para o segundo time**

**Acessar a pasta do time *UaiSoccer2D***

**Figura 8:** Comando para acessar pasta

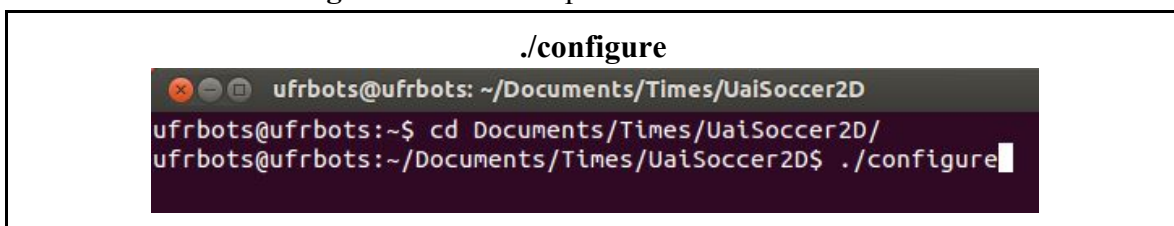


Fonte: Próprio autor

**Executar arquivo base do time**

Após acessar a pasta, pelo passo anterior, deve-se executar o arquivo base do time para compor predefinições.

**Figura 9:** Comando para executar time base



Fonte: Próprio autor

### Compilar time

O passo seguinte é muito importante e deve ser executado em seguida ao passo anterior. Aqui o time será compilado para ser iniciado.

**Figura 10:** Comando para compilar arquivos do time

```
make  
ufrbots@ufrbots: ~/Documents/Times/UaiSoccer2D  
ufrbots@ufrbots:~/Documents/Times/UaiSoccer2D$ make
```

Fonte: Próprio autor

### Colocar time em campo

A próxima etapa se resume em iniciar o time, ou seja, prepará-lo em campo no simulador para uma nova partida. Dessa forma o time *UaiSoccer2D* entra em campo junto com o *RoboCore1* e juntos aguardam o início da partida.

**Figura 11:** Comando para iniciar time

```
./start.sh  
ufrbots@ufrbots: ~/Documents/Times/UaiSoccer2D  
ufrbots@ufrbots:~/Documents/Times/UaiSoccer2D$ ./start.sh
```



Fonte: Próprio autor

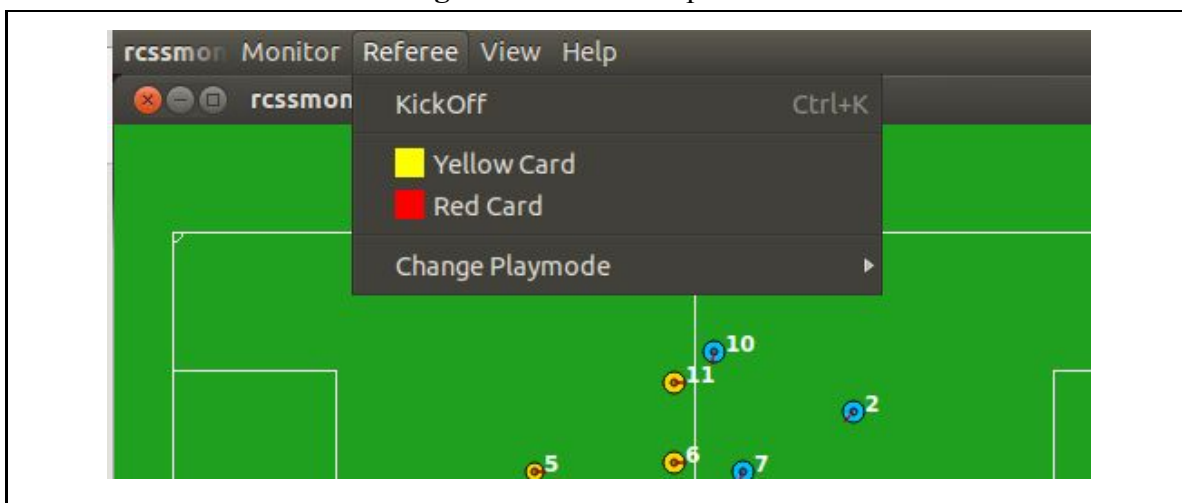
Observe que cada time possui sua formação inicial e por convenção o próprio simulador define a cor dos jogadores de cada time, o que entra primeiro em campo na cor amarelo e o segundo na cor azul, como também de costume o placar inicial é 0x0 e o próximo passo será iniciar a partida.

### Iniciar partida

Estando os dois times que vão disputar uma partida em campo, o próximo passo será executado diretamente no simulador, onde no comando *kickOff*, a partida será iniciada, conforme figura abaixo.

*Referee > kickOff* (para iniciar partida)

**Figura 12:** Iniciar de partida



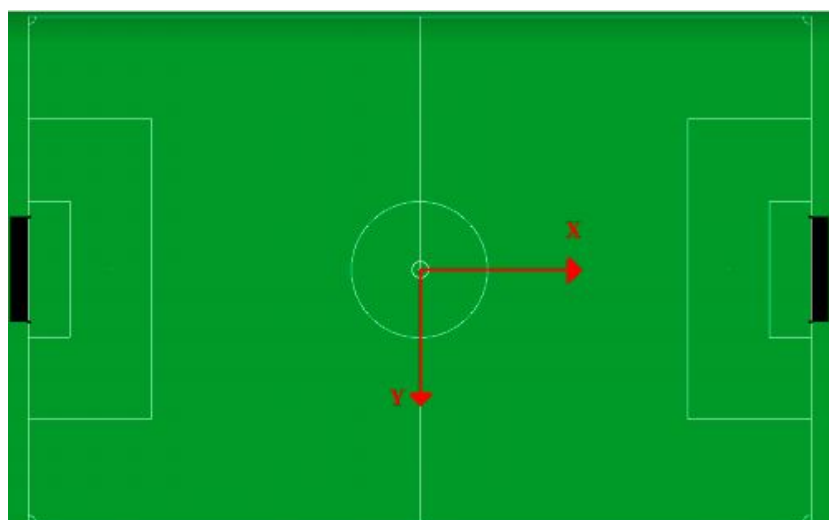
Fonte: Próprio autor

|                                                                       |
|-----------------------------------------------------------------------|
| UFRBots                                                               |
| Uma visão extensionista para o futebol de robôs no Recôncavo da Bahia |
| <b>Aula 5 - 2ª Parte - Comandos 2D</b>                                |

## Formação

A origem do sistema de coordenadas é o centro do campo, sendo que a sua orientação é relativa ao campo de ataque do time. Se o seu time está atacando para a direita, então a coordenada x positiva estará apontada para a direita e a coordenada y positiva estará apontada para baixo. Caso o seu time esteja atacando para a esquerda, então a coordenada x positiva estará direcionada para a esquerda e a coordenada y positiva estará direcionada para cima. Isso quer dizer que você não precisa fazer dois códigos, um para quando o seu time estiver do lado esquerdo e um para quando o seu time estiver no lado direito.

**Figura 1:** coordenadas do campo.



Fonte: coordenadas do campo. (Oliveira, 2011).

**Observação:** O campo varia em X de 52,5 a -52,5 e em Y de 36 a -36.

## Mudança de Formação

Para alteração de formação deve se procurar a seguinte linha do código:

### Caso:1

Formação Inicial:

formations->setFormation( FT\_INITIAL );

**Observação:** Altere apenas o que está em negrito.

Opção para Alteração:

- FT\_433\_OFFENSIVE
- FT\_343\_OFFENSIVE
- FT\_DEFENSIVE
- FT\_OPEN\_DEFENSIVE
- FT\_343\_ATTACKING

**Exemplo:** Mude a formação inicial para FT\_433\_OFFENSIVE .

**Antes:**

Formação no código (no arquivo src/PlayerTeams.cpp, linha 93)

**Figura 2:** FT\_INITIAL.

```
if( formations->getFormation() != FT_INITIAL || // not in kickoff formation
    posAgent.getDistanceTo( WM->getStrategicPosition() ) > 2.0 )
{
    formations->setFormation( FT_INITIAL );    // go to kick_off formation
    ACT->putCommandInQueue( soc=teleportToPos( WM->getStrategicPosition() ));
}
```

Fonte: Próprio autor.

- Formação no simulador antes da alteração.

**Figura 3:** formação Inicial no simulador.



Fonte: Próprio autor.

Depois:

**Figura-4:** Alteração de formação inicial.

```
formations->setFormation(FT_433_OFFENSIVE);
soc.commandType = CMD_ILLEGAL;
if( WM->getConfidence( OBJECT_BALL ) < PS->getBallConfThr() )
{
    ACT->putCommandInQueue( soc = searchBall() ); // if ball pos unknown
    ACT->putCommandInQueue( alignNeckWithBody( ) ); // search for it
}
```

Fonte: Próprio autor.

**Figura 5:** Mudança de formação.



Fonte: Próprio autor.

## Caso: 2

Formação depois do início do jogo:

```
formations->setFormation(FT_DEFENSIVE);
```

**Observação:** Altere apenas o que está em negrito.

Opção para Alteração:

- FT\_433\_OFFENSIVE;
- FT\_343\_OFFENSIVE;

- FT\_DEFENSIVE;
- FT\_OPEN\_DEFENSIVE;
- FT\_343\_ATTACKING;

### Exercícios:

- 1) Mude a uma formação defensiva simule o jogo contra os Reservas.
- 2) Mude para uma formação ofensiva e análise os resultados para alterações nestas formações.

### Criar própria formação:

Para editar a formação e a posição de cada agente no campo ao seu querer é possível modificando o arquivo *formations.conf* alterando a estruturas das formações acima ou até criando novas.

**Figura 6:** posições dos agentes.

```
# Formation 1 = FT_initial = 111
1
-55.0 -14.0 -17.0 -17.0 -14.0 -12.0 -5.0 -5.0 -2.0 0.0 0.0 # X_pos
0.0 16.0 5.0 -5.0 -16.0 0.0 12.0 -12.0 0.0 22.0 -22.0 # Y_pos
1 4 2 2 4 5 6 6 8 7 7 # P_type
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 # X_attr
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 # Y_attr
0 0 0 0 0 0 0 0 0 # Behind_ball
-49.0 -45.0 -45.0 -45.0 -45.0 -45.0 -45.0 -45.0 -45.0 # X_min
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 # X_max
```

Fonte: Próprio autor.

Cada coluna representa um agente, ou seja, um jogador que segue a seguinte ordem, pensando em uma formação 433 a distribuição ficaria o seguinte:

**Tabela 1:** Distribuição da formação 433.

| Coluna         | Jogador(posição) | Número do agente |
|----------------|------------------|------------------|
| 1 <sup>a</sup> | Gk- Goleiro      | 1                |
| 2 <sup>a</sup> | LT- Lateral      | 2                |
| 3 <sup>a</sup> | ZG- Zagueiro     | 3                |
| 4 <sup>a</sup> | ZG- Zagueiro     | 4                |

|                 |                |    |
|-----------------|----------------|----|
| 5 <sup>a</sup>  | LT- Lateral    | 5  |
| 6 <sup>a</sup>  | MC- Meio Campo | 6  |
| 7 <sup>a</sup>  | MC- Meio Campo | 7  |
| 8 <sup>a</sup>  | MC- Meio Campo | 8  |
| 9 <sup>a</sup>  | AT- Atacante   | 9  |
| 10 <sup>a</sup> | AT- Atacante   | 10 |
| 11 <sup>a</sup> | AT- Atacante   | 11 |

Esta posição de atuação dos jogadores pode ser mudada conforme a edição da formação.

**Exemplo:** Edite o posicionamentos dos agentes 3,4 e 6 na FT\_INICIAL:

**Antes:**

**Figura 7:** Formação inicial.

```
# Formation 1 = FT_initial = 111
1
-55.0 -14.0 -17.0 -17.0 -14.0 -12.0 -5.0 -5.0 -2.0 0.0 0.0 # X_pos
0.0 16.0 5.0 -5.0 -16.0 0.0 12.0 -12.0 0.0 22.0 -22.0 # Y_pos
1 4 2 2 4 5 6 6 8 7 7 # P_type
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 # X_attr
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 # Y_attr
0 0 0 0 0 0 0 0 0 # Behind_ball
-49.0 -45.0 -45.0 -45.0 -45.0 -45.0 -45.0 -45.0 -45.0 # X_min
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 # X_max
```

Fonte: Próprio autor.

**Figura 8:** FT\_INICIAL.



Fonte: Próprio autor.

Depois:

Figura 9: Nova formação.

```
# Formation 1 = FT_initial = 111
1
-55.0 -14.0 -28.0 -28.0 -14.0 -25.0 -5.0 -5.0 -2.0 0.0 0.0 # X_pos
0.0 16.0 5.0 -5.0 -16.0 0.0 12.0 -12.0 0.0 22.0 -22.0 # Y_pos
1 4 2 2 4 5 6 6 8 7 7 # P_type
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 # X_attr
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 # Y_attr
0 0 0 0 0 0 0 0 0 # Behind_ball
-49.0 -45.0 -45.0 -45.0 -45.0 -45.0 -45.0 -45.0 -45.0 # X_min
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 # X_max
```

Fonte: Próprio autor.

Figura-10: Alteração da formação inicial.



Fonte: Próprio autor

**Observação:** É importante lembramos que depois de cada alteração na formação devemos Salvar e dar um **make** para modificar a formação do time.

#### Exercícios:

- 1) Construa uma formação defensiva para encarar equipes muito ofensivas e simule o jogo.
- 2) Crie uma formação ofensiva e simule as duas que você construiu contra o time dos Reservas ou seus colegas e analise os resultados e definam algumas estratégia final para o campeonato.

## UFRBots

Uma visão extensionista para o futebol de robôs no Recôncavo da Bahia

### Aula 6 - Comandos 2D

#### Chute

```
soc = kickTo(posGoal, SS->getBallSpeedMax());
```

**Exemplo:** (no arquivo `src/PlayerTeams.cpp`, linha 184)

- Chute para Gol:

A estratégia do Time Base é chutar para o gol sempre que a bola estiver na área de chute do jogador.

**Figura 1:** Chute a gol.

```
if((distl>2)&&(posAgent.getY()<40)){//verifica a margem segura
    if(posAgent.getX()<10){//posicao em x menor q 10
        soc = dribble(0, DRIBBLE_FAST);//liga o drible
    }else{ if(posAgent.getX()<20){
        soc = directPass(posAgent_passe10, PASS_FAST);//passe p 10
    }else{
        soc = kickTo( posGoal, SS->getBallSpeedMax() ); //chuta p gol
    }
}
}else{
    if(posAgent.getX()<30){//posicao em x menor que 30
        soc = directPass(posAgent_passe10, PASS_FAST);//passe p 10
    }else{
        soc = kickTo( posGoal, SS->getBallSpeedMax() ); //chuta p gol
    }
}
```

Fonte: Próprio autor.

#### Tipos de Drible

- Lento:

```
soc = dribble(0, DRIBBLE_SLOW);
```

- Rápido:

```
soc = dribble(0, DRIBBLE_FAST);
```

- Outro:

```
soc = dribble(0, DRIBBLE_WITHBALL);
```

#### Passes:

- Normal:

```
soc = directPass(posAgent_passe8, PASS_NORMAL);
```

- Rápido:

```
soc = directPass(posAgent_passe10, PASS_FAST);
```

#### Exemplos:

- Escolhendo jogador para Chutar ou Driblar:  
(no arquivo src/PlayerTeams.cpp, linha 450).

**Figura 2:** Seleção para chute.

```
else if( WM->isBallKickable() )  
{  
    soc = clearBall(CLEAR_BALL_DEFENSIVE, SIDE_ILLEGAL, NULL);  
    ACT->putCommandInQueue( soc );  
    ACT->putCommandInQueue( turnNeckToObject( OBJECT_BALL, soc ) );  
}
```

Fonte: Próprio autor.

- Verificando posição de X para chute:(no arquivo src/PlayerTeams.cpp, linha 228).

**Figura 3:** Verificando posição.

```
}else{ if(posAgent.getX()<35){  
    soc = dribble(0, DRIBBLE_SLOW);  
}else{if(posAgent.getX()<posAgent_passe11.getX()-1){  
    soc = directPass(posAgent_passe11, PASS_FAST);  
}else{  
    soc = kickTo( posGoal, SS->getBallSpeedMax() );//chuta  
}  
}  
}
```

Fonte: Próprio autor.

```

if((distl>2)&&(posAgent.getY()<40)){//verifica a margem segura
    if(posAgent.getX()<9){//posicao em x menor q 10
        soc = dribble(0, DRIBBLE_SLOW);//liga o drible
    }else{ if(posAgent.getX()<15){
        soc = directPass(posAgent_passe9, PASS_FAST);//passe p 10
    }else{
        soc = kickTo( posGoal, SS->getBallSpeedMax() ); //chuta p gol
    }
}
}else{
    if(posAgent.getX()<25){//posicao em x menor que 25
        soc = directPass(posAgent_passe9, PASS_FAST);//passe p 9
    }else{
        soc = kickTo( posGoal, SS->getBallSpeedMax() ); //chuta p gol
    }
}
}

```

### Exercícios:

- 1) Desenvolver uma estratégia com as seguintes características em que os Jogadores:
  - 2,3,4 e 5 carreguem a bola devagar;
  - 6 e 7 carregue a bola rápido;
  - 9,10 e 11 chutem a bola no gol;
- 2) Desenvolver uma estratégia com formação 334, em que os jogadores chutam a bola caso posição  $x < 0$  ou  $x > 30$ . Caso contrário o agente carrega a bola.
- 3) Alterar a formação no arquivo formations.conf de acordo com a tabela abaixo.

```
# Formation 3 = FT_334_offensive
3
-50.0 -25.0 -25.0 -10.0 -30.0 3.0 3.0 15.0 15.0 18.0 18.0 # X_pos
0.0 5.0 0.0 0.5 -5.0 -5.0 11.0 -5.0 5.0 19.0 -19.0 # Y_pos
1 4 2 5 4 6 6 8 8 7 7 # P_type
0.0 0.1 0.6 0.6 0.6 0.6 0.6 0.5 0.6 # X_attr
0.0 0.1 0.25 0.25 0.25 0.25 0.3 0.2 0.2 # Y_attr
0 1 1 1 0 0 0 0 0 #Behind ball
0.0 -50.5 -45.0 -45.0 -45.0 -30.0 -30.0 -45.0 -2.0 # X_min
0.0 -30.0 0.0 0.0 2.0 20.0 42.0 44.0 45.0 # X_max
```

## O que aconteceu?

- 4) Crie uma formação e estratégia que valoriza a posse de Bola. Seu time deve valorizar a bola mais que o seu adversário.

|                                                                       |
|-----------------------------------------------------------------------|
| UFRBots                                                               |
| Uma visão extensionista para o futebol de robôs no Recôncavo da Bahia |
| <b>Aula 7 - 1ª Parte - Preparação dos Times</b>                       |

## Nome do Time

Decida e defina o nome do seu time alterando no arquivo *start.sh*

## Formação

Para a montagem do seu time base para a competição final, são necessárias algumas definições básicas que vão definir o resultado do jogo. Lembrando que cada decisão tomada, vai impactar diretamente em todo o comportamento e performance do seu time.

Como passo inicial será necessário iniciar a definição de formação do time, que podem ser:

- I. Formação inicial
- II. Formação quando o time adversário possui a bola
- III. Formação quando seu time possui a bola

Exemplos básicos de formação:

- I. FT\_433\_OFFENSIVE
- II. FT\_343\_OFFENSIVE
- III. FT\_DEFENSIVE
- IV. FT\_OPEN\_DEFENSIVE
- V. FT\_343\_ATTACKING

Lembrando que se preferir você pode criar sua própria formação

## Chute

O chute pode ser definido para os seguintes fatores:

- I. Distância
  - A. Do gol
  - B. Do agente para o gol
- II. Força
- III. Ângulo

Tipos de chute:

- I. Rápido

## II. Lento

### Passe

- I. Normal
- II. Rápido

### Jogar Partida

As alterações acima, permitem uma evolução considerável do time, dessa forma, o próximo passo será realizar uma partida com o time base reserva do UaiSoccer2D. Com a pré-definição dos resultados observados de seu time, realize novas alterações quando achar necessário.

|                                                                       |
|-----------------------------------------------------------------------|
| UFRBots                                                               |
| Uma visão extensionista para o futebol de robôs no Recôncavo da Bahia |
| <b>Aula 8 - 2ª Parte - Preparação dos Times</b>                       |

## Parâmetros do Time

Um time como simulação real, possui desgaste energético que culmina diretamente no rendimento dos jogadores ao longo da partida. Lembre-se desse ponto durante as alterações básicas do time, pois cada decisão tomada interfere diretamente no quão rápido um jogador vai ficar cansado, onde estratégias que intuitivamente exercem um desgaste físico maior pode trazer consequências negativas para o seu time ao jogar o segundo tempo.

No arquivo *player.conf* é possível ajustar a distância da marcação, onde é possível alterar a distância:

- I. Marcação
  - A. Próximo
  - B. Distante
- II. Ângulo do chute
  - A. turn\_with\_ball\_ang\_thr
- III. Distância de tolerância
  - A. player\_dist\_tolerance
- IV. Distância para bola
  - A. clear\_ball\_dist
- V. Velocidade do passo
  - A. fast\_pass\_end\_speed

## Jogar Partida

As alterações acima juntamente com a aula anterior, permitem uma evolução considerável do time, contudo essa próxima partida não será contra o time base do UaiSoccer2D. Inicie uma partida com o time adversário do seu instrutor e avalie os resultados da partida para que sejam realizadas modificações considerando as aulas vistas.

### **Sorteio do Campeonato Final**

Como predefinição da competição final, será realizado o sorteio dos times que vão se enfrentar nas primeiras partidas. Definições iniciais para sorteio:

- I. Os times de um mesmo instrutor não vão se enfrentar na fase de grupo.
- II. Serão dos classificados na fase de grupo

|                                                                       |
|-----------------------------------------------------------------------|
| UFRBots                                                               |
| Uma visão extensionista para o futebol de robôs no Recôncavo da Bahia |
| <b>Instalação da Máquina Virtual</b>                                  |

Nesta seção, serão dispostos os passos necessário para a instalação da máquina virtual, da qual possibilita a aplicação de todo conteúdo expresso na apostila.

### Download e Instalação do VirtualBox

- 1) Faça o Download do VirtualBox
  - a) Link para Download: <https://www.virtualbox.org/>
  - b) Em seguida siga os passos da instalação da Virtual box;

**Figura 1:** Máquina virtual box.



Fonte: Próprio autor.

### Download e Instalação da Máquina Virtual UFRBots

- 1) Faça o Download da máquina Virtual com o simulador e dependências. Já está incluso na pasta *Documents*, o arquivo do time base *Uaisoccer2D*.
- 2) Link para download:  
<https://drive.google.com/file/d/1URQxlRdnK26CR2PkEuL3QkRBQw9QZdB4/view?usp=sharing>

- 3) Ao término do Download abra o Virtualbox
- a) Selecione Importar como mostra a imagem a seguir:

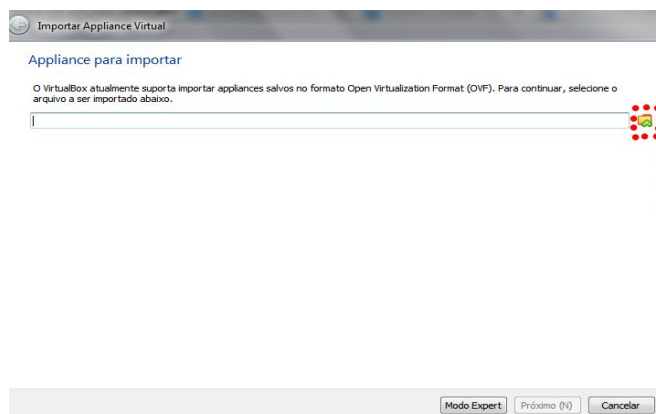
**Figura 2:** Instalação UFRBots.



Fonte: Próprio autor.

- b) Selecione a Pasta onde foi salvo o Download da Máquina virtual da UFRBots. Busque clicando nas pasta lateral como na imagem a seguir. Depois de selecionada a pasta click em **Próximo (N)**.

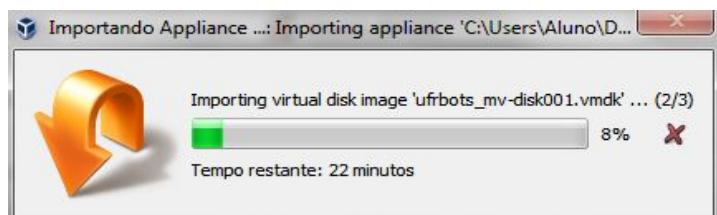
**Figura 3:** Seleção de Máquina Virtual.



Fonte: Próprio autor

- c) Em seguida Mantenha todas as configurações como estão e clique em **importar** e aguarde o tempo para importação.

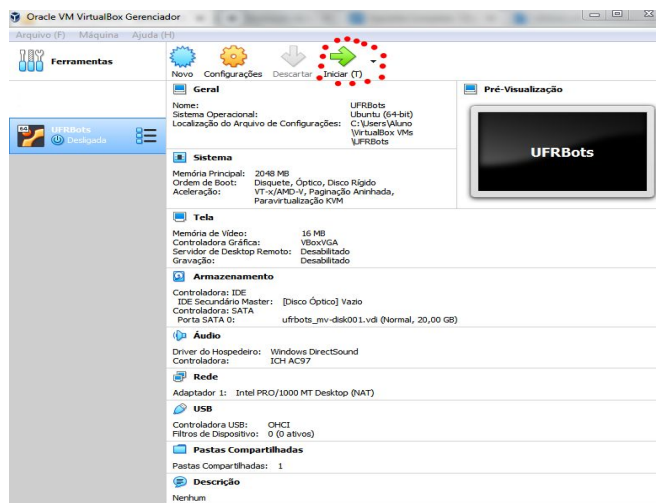
**Figura 4:** Importação da Máquina Virtual



Fonte: Próprio autor.

d) Agora click em iniciar como indica a figura abaixo:

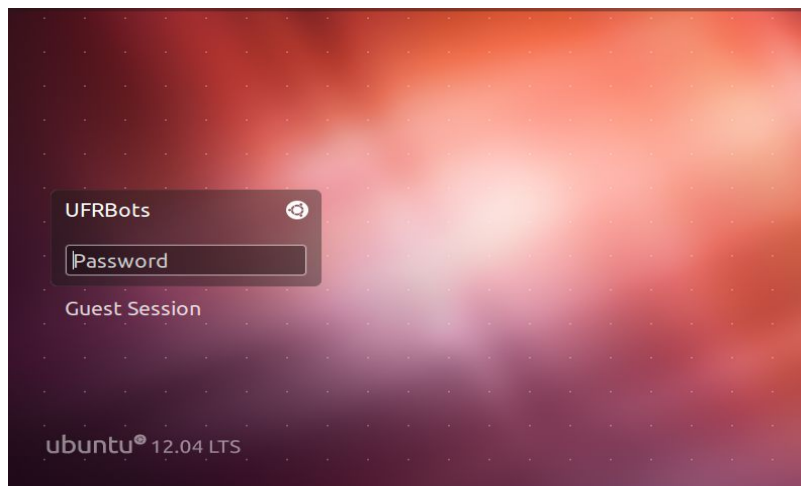
**Figura 5:** Iniciando Máquina virtual.



Fonte: Próprio autor.

e) Pronto instalação feita com Sucesso a Senha para Login é: **ufrbots**.

**Figura 6:** Login UFRBots



Fonte: Próprio autor.

## Referências

- [1] Ottoni, A. L. C. (2010). Introdução à robótica. *Material de estudo. I ORCV Olimpíada de Robótica do Campo das Vertentes, Universidade Federal de São João del Rey.*
- [2] de Souza Pio, J. L., de Castro, T. H. C., & de Castro Júnior, A. N. (2006, November). A robótica móvel como instrumento de apoio à aprendizagem de computação. In *Brazilian Symposium on Computers in Education (Simpósio Brasileiro de Informática na Educação-SBIE)* (Vol. 1, No. 1, pp. 497-506).
- [3] SILVA, R. D., Santos Filho, D. J., & Miyagi, P. E. (2015). Modelagem de Sistema de Controle da Indústria 4.0 Baseada em Holon, Agente, Rede de Petri e Arquitetura Orientada a Serviços. XII Simpósio Brasileiro de Automação Inteligente. Natal.
- [4] VELLOSO, F.C. Informática: conceitos básicos. Rio de Janeiro: Editora Campus, 1999.
- [5] FEDELI, R.D I Introdução à Ciência da Computação, Ed. Thomson, 2003 .
- [6] TOCCI, R.J. Sistemas Digitais: princípios e aplicações Prentice Hall, 2003.
- [7] MANZANO. Algoritmos: Lógica de Programação de Computadores. Ed. Érica, 2000.
- [8] DEITEL. Java como programar. Prentice Hall, 2005.
- [9] CUNHA, R.D. Introdução à Linguagem de Programação Fortran 90.
- [10] SANTOS, V. S., SUPERLOGO, Programação para o estudo de geometria. Bauru, Sp: UNESP, 2006. <http://wwwp.fc.unesp.br>
- [11] FACCAROLI, E. S.(2009) Ambiente de futebol simulado 2D, Universidade Federal de São João Del Rey.
- [12] RoboCup. (2013), RoboCupSoccer simulation 2D, Manual para categoria de futebol de Robos simulação 2D.
- [13] BOER J. R. Kok, R.: The incremental development of a synthetic multi- agent system: The uva trilearn 2001 robotic soccer simulation team. Master's thesis, University of Amsterdam, The Netherlands.
- [14] Vaz, M. Apostila de apresentação a Linguagem C.
- [15] Rebollo, C.(2013) Centro Nacional de Alto Desempenho. CENAPAD-SP. Introdução a Linguagem C.
- [16] (Universidade Federal de Minas Gerais Instituto de Ciências Exatas Departamento de Ciência da Computação).

